

Porting SAWS to GPU Accelerators

Design notes on dynamic load balancing for irregular workloads on CUDA/NVSHMEM systems

D. Brian Larkins

August 2026

Contents

1	What SAWS Actually Contributes	2
2	The Approach Space for GPU Irregularity	3
2.1	Flattening / load-balanced search	3
2.2	Persistent threads	3
2.3	Warp-level redistribution	3
2.4	Device-side launch	3
2.5	Megakernel task schedulers	3
3	Where SAWS Ports	4
3.1	NVSHMEM as the enabling substrate	4
3.2	A hierarchical mapping	4
3.3	What must be redesigned rather than ported	4
4	The Genuinely New Problem	6
4.1	Secondary consequences	6
5	Termination Detection	7
6	Recommended First Prototype	8
6.1	Evaluation framing	8
7	Appendix: Prototype	9

1 What SAWS Actually Contributes

Before asking how the system ports, it is worth isolating what in it is genuinely load-bearing. The transferable asset is not the ring buffer.

In `saws_shrb.c`, a steal is four operations:

1. one `shmem_atomic_fetch_add(&steal_val, 1L<<40, proc)`
2. some purely local arithmetic
3. one `shmem_getmem_nbi()` directly out of the victim's buffer
4. one `shmem_atomic_add()` to report completion

The victim is never involved. There is no lock, no progress thread on the victim, no round trip beyond the single atomic.

This works because the entire negotiable state of the queue is packed into one 64-bit word:

Field	Bits	Meaning
<code>asteals</code>	40 - 63	steals claimed so far in this epoch
<code>epoch</code>	38 - 39	epoch index / validity marker
<code>itasks</code>	19 - 37	tasks present when the epoch opened
<code>vtail</code>	0 - 18	base ring index for the epoch

and because the allocation schedule is **deterministic**. The fetch-add returns the thief's ticket *and* the epoch parameters simultaneously. The thief then replays the geometric halving schedule locally to derive exactly which byte range belongs to it. Every thief computes the same answer independently, with no further communication.

The principle, stated generally:

Replace negotiation with a locally computable allocation function over an atomically issued ticket. Spend arithmetic to buy away communication.

That tradeoff is **more** favorable on a GPU than on a cluster, not less. ALU throughput is nearly free; global atomics serialize under contention, and cross-block coordination carries no forward-progress guarantee. A protocol whose fast path is one atomic plus pure arithmetic is close to the ideal shape for GPU hardware.

2 The Approach Space for GPU Irregularity

Existing approaches divide naturally by *when* the imbalance becomes known.

2.1 Flattening / load-balanced search

Prefix-sum the per-item work counts, then binary-search the scan to assign equal output ranges to threads (CUB, ModernGPU merge-path). This is optimal when work per item is knowable before launch, and it is what most production irregular GPU kernels actually do.

It is useless for discovery-driven irregularity, where the work is only revealed by executing the work. That is exactly the workload class SAWS targets.

2.2 Persistent threads

Launch exactly enough blocks to fill the device, then loop pulling from a global queue. Aila and Laine established this pattern for ray tracing. The kernel outlives any individual work item, so scheduling becomes a software concern rather than a hardware one.

This is the family SAWS belongs to, and the one to target.

2.3 Warp-level redistribution

Ballot- and shuffle-based donation among lanes, warp-aggregated atomics. Very cheap, but only addresses intra-warp imbalance. Necessary but not sufficient.

2.4 Device-side launch

CUDA Dynamic Parallelism. Historically dominated by launch overhead; CDP2 in CUDA 12 improved the situation, but it remains uncompetitive with a good persistent-kernel scheduler at fine task granularity.

2.5 Megakernel task schedulers

Whippletree, Softshell. Closest in spirit to the existing task collection: typed task descriptors, device-resident queues, dispatch by task class. These demonstrate that the programming model survives the move to GPU; they do not solve distributed balancing.

3 Where SAWS Ports

3.1 NVSHMEM as the enabling substrate

NVSHMEM exposes the OpenSHMEM API to device code. `nvshmem_uint64_atomic_fetch_add` and `nvshmem_getmem_nbi` are callable from inside a running kernel, and warp- and block-scoped variants (`nvshmemx_getmem_nbi_warp`) let a whole warp cooperate on a single transfer.

This means the steal protocol can move into a persistent CUDA kernel nearly verbatim, substituting a warp or block for a PE. That is an unusually short path from an existing, debugged, published protocol to a working GPU system. Most efforts to port work stealing to GPUs must first invent the communication layer.

3.2 A hierarchical mapping

The cost of a steal varies by roughly four orders of magnitude across levels, so the balancer should be tiered, with the cheap tiers absorbing most of the imbalance.

Level	Steal mechanism	Approx. cost
Lane to lane	<code>__shfl_sync</code> / ballot redistribution	free, no atomics
Warp to warp	ticket word in shared memory	L1 latency
Block to block	SAWS ticket word in global memory	L2 atomic
GPU to GPU	NVSHMEM device atomics over NVLink	microseconds
Node to node	existing SAWS protocol, unchanged	microseconds+

The top two tiers are the existing system. The bottom three are new, but they run the same protocol with different backing memory and different atomic scope.

3.3 What must be redesigned rather than ported

The geometric schedule must be abandoned, not merely retuned. There are two separate problems and only the first is obvious.

Claimant count. `SAWS_MAX_STEALS_PER_EPOCH` is 22 because halving from N terminates in roughly $\log_2 N$ claims. With 132 SMs and several resident warps each, thousands of potential claimants compete for 22 slots.

Share size. Halving front-loads enormous shares — the first thief is offered half the epoch. A CPU thief accepts that into a `malloc`'d buffer. A GPU thief cannot: a warp has a fixed staging buffer and executes 32 tasks at a time. It must therefore either truncate its claim or loop. **Truncating is silently incorrect**, because later tickets derive their offsets from the schedule rather than from what any thief actually transferred, so the untaken remainder of an over-large claim is orphaned and never executed by anyone.

This rules out the tempting intermediate design — halve until the share drops below a warp, then issue fixed chunks — which repairs the claimant count but leaves the first ticket claiming 131072 tasks out of a 256K epoch.

The schedule that works is flat. Divide the epoch into fixed chunks of one warp and let a thief claim as many *consecutive* chunks as it can hold, by adding that many to the claim counter in its single atomic:

$$\text{offset}(k) = k \cdot C, \quad \text{share}(k) = \min(n \cdot C, N - \text{offset}(k))$$

Epoch size	geometric claimants	flat claimants
1 024	6	32
16 384	10	512
262 144	14	8 192

This is still $O(1)$ — a multiply and a subtract, with no loop and no divergence, which is itself an improvement on the $O(\log N)$ victim-replay in `saws_shrb.c:666`. What is surrendered is steal-half adaptivity, which mattered on the CPU because a steal cost a network round trip. The equivalent lever becomes the chunk count n , chosen per tier from known latency rather than derived from queue occupancy: a remote steal claims several chunks, a local one claims just enough.

One implementation note that cost real time to find. The invariant a schedule must satisfy is that the shares tile the epoch exactly — no gap, no overlap, and zero for any ticket past the end. Any geometric formulation expressed as $\text{share}(k) = N \gg (k+1)$ violates it for every odd N , because integer truncation breaks the telescoping; it has to be written as the difference of consecutive remainders, $(N \gg k) - (N \gg (k+1))$. On a GPU that bug surfaces as a handful of tasks silently never executing, on odd epoch sizes only. Whatever schedule is adopted, the tiling invariant is worth checking exhaustively over small N before any of it runs on hardware.

The completion array is the wrong shape. `completed[epoch].status[22]` assumes few, individually identifiable thieves. At GPU scale the right structure is a pair of counters per epoch — tickets retired and tasks taken — with the epoch closed by a `fetch_or` that returns the final ticket count. Reclamation then tests `retired == issued` rather than scanning an array. This is $O(1)$ state per epoch instead of $O(\text{maxsteals})$.

Local operations are no longer synchronization-free. On the CPU the owner manipulates the head with no atomics at all. On the GPU the “owner” is an entire block, so head manipulation requires block-scoped atomics (`atomicAdd_block`) in shared memory. This is still cheap — roughly L1 latency, no cross-SM traffic — but it is not free, and it changes the accounting for very fine tasks.

4 The Genuinely New Problem

On a CPU, work stealing optimizes a single objective: load. On a GPU it must jointly optimize load **and SIMD coherence**.

A warp that steals a mixed bag of task types diverges on dispatch and loses most of its throughput. It is entirely possible to balance the load perfectly and still lose, because the gain from balancing is smaller than the loss from divergence. The two objectives are in direct tension: the most available work is frequently the wrong *kind* of work.

This reframes the data structure. The queue is not a deque of tasks; it is a set of per-task-class bins, and the steal decision must weigh “who has work” against “who has work of the class my warp is already executing.”

The existing LAWS variant already contains the machinery for a steal policy driven by something other than raw availability — locality there, task-class coherence here. It is the same structural change measured against a different metric.

This is where the research contribution lies. It is also the piece none of the existing GPU task schedulers handle well: Whippletree bins by class but does not balance across devices; SAWS balances across devices but does not bin.

4.1 Secondary consequences

- **Steal granularity must be a multiple of the warp width.** A steal that returns 3 tasks wastes 29 lanes. This falls out of the flat schedule above for free, since the chunk *is* a warp.
- **Contiguous steals are coalescable.** SAWS already transfers `ntasks * elem_size` contiguous bytes in a single `getmem`. Mapped onto a warp-collective transfer, this is a perfectly coalesced read. The existing layout is accidentally ideal for GPU.
- **Task descriptors must be small and POD.** No pointers into host memory, no function pointers used across the host/device boundary without a registry indirection.
- **Forward progress.** The steal path must be lock-free. It already is — the `synch_mutex_t` in `saws_shrb_s` is not on the fast path. This is a significant asset, because a lock-based scheme risks deadlock whenever co-residency of blocks is not guaranteed.

5 Termination Detection

The tree-based token termination in `termination.c` transfers with only a change of scope, but a GPU system needs it at two levels.

Within a device, termination is cheap: a global count of active blocks, decremented when a block finds no work anywhere and incremented when it receives work. A grid-wide barrier via cooperative groups can confirm quiescence. No token passing is required, because all blocks share coherent global memory.

Across devices and nodes, the existing tree token applies directly. The only change is that the vote must be initiated from device code, or the persistent kernel must periodically yield to a host-side progress engine. The first is cleaner; the second is far easier to debug and is the right choice for a prototype.

The spawned/completed counter pair that drives the vote maps to two device atomics. The subtlety is that a task counted as spawned on one device may be executed on another, exactly as in the current system, so the existing correctness argument carries over unchanged.

6 Recommended First Prototype

Take UTS. It is already in `examples/uts`, published GPU UTS results exist for comparison, and it is pure discovery-driven irregularity with a trivial task descriptor. That isolates the load-balancing question from the coherence question entirely.

Phase 1. Single GPU, persistent kernel, block-level ticket deque with a fixed schedule function. Target: match a hand-tuned UTS GPU implementation.

Phase 2. Add the NVSHMEM tier for multi-GPU, reusing the existing termination detection at the top level. Target: strong scaling across a node.

Phase 3. Introduce class-binned queues and a coherence-aware steal policy. Evaluate on BPC, which has two task classes with a 10x cost ratio and is therefore a much better vehicle for this question than UTS.

Phase 4. Multi-node. At this point the existing SAWS code is doing the top-level balancing unmodified, and the contribution is the composition rather than any single tier.

6.1 Evaluation framing

The interesting number is not raw throughput against a hand-tuned kernel; it is the *breadth of the regime* in which the dynamic scheduler wins. Static flattening wins when work is predictable. The claim to defend is that as irregularity increases — measured by the variance of per-task cost, or the depth variance of the task tree — there is a crossover beyond which dynamic balancing dominates, and that the crossover occurs at a level of irregularity that real applications actually exhibit.

That framing also makes BPC the more valuable benchmark, since its producer / consumer work ratio is a directly tunable irregularity knob.

7 Appendix: Prototype

A code sketch of the above lives in `libgputc/`, with a port of BPC in `gpubpc/`. Neither has been compiled — there is no CUDA-capable device on the machine they were written on — and they exist to make the structure concrete rather than to run. `libgputc/README.md` records the design decisions and an honest list of gaps.

Mapping to the existing tree:

Prototype	libtc equivalent
<code>gputc_types.cuh</code>	<code>saws_shrb.h</code> , <code>tc.h</code>
<code>gpu_shrb.cuh</code>	<code>saws_shrb.c</code>
<code>gpu_task.cuh</code>	<code>task.c</code>
<code>gpu_termination.cuh</code>	<code>termination.h</code>
<code>collection.cu</code>	<code>collection-saws.c</code>
<code>gputc_host.cu</code>	<code>init.c</code> , <code>handle.c</code>

The single most encouraging detail to come out of writing it: the intra-device and inter-PE steal tiers run *identical code*. `gputc_warp_steal()` chooses between a plain `atomicAdd` and `nvshmem_uint64_atomic_fetch_add` on whether the victim is peer-visible, and nothing else about the protocol changes. That the same handful of lines serves both an L2 atomic and an RDMA operation is good evidence that the ticket abstraction is at the right level.

The least encouraging: the allocation schedule needed three attempts before it was correct, and the failure mode of the wrong versions was tasks silently disappearing rather than anything that would announce itself. Whatever else the port inherits from the CPU system, it should also inherit a test for the tiling invariant.